Data Lifecycle

Explain what happens to the data in each stage of the Data Lifecycle.

Lifecycle Stage	What Happens to the Data
Creation	Cashiers, Inventory Managers, and Admins input data through the Vercel-hosted frontend web interface (creating bicycle products/SKUs, uploading product images, recording stock shipments, and processing retail POS checkout orders). System activity logs and customer sales records are automatically generated upon each event.
Storage	Transactional and relational business records are stored securely in a managed cloud PostgreSQL database hosted on **Supabase** (Singapore `ap-southeast-1`). Uploaded product media and shop assets are stored in **Cloudinary** cloud storage (`ihlkdz00`). Sensitive user passwords are irreversibly hashed using `bcrypt` (12 rounds) before database storage.
Usage	Serverless PHP execution runtimes on **Vercel** (Singapore `sin1`) query inventory levels, execute atomic stock decrements upon checkout completion, compute subtotal and 12% VAT calculations, and aggregate financial analytics (Gross Revenue, COGS, Net Profit) displayed on live dashboards. Database calls utilize Supabase's high-speed Transaction Pooler (Port 6543).
Sharing	Authorized users access processed data through strictly defined role-based interfaces. Generated customer sales invoices, receipt slips, and business summary reports (PDF/CSV) are delivered securely over HTTPS (TLS 1.3) via Vercel Edge CDN, while product images are served globally through Cloudinary CDN URLs.
Archiving	Past transaction ledgers, closed sales orders, and audit trail logs stored in Supabase are periodically backed up into compressed SQL dumps and CSV exports. Cloudinary automatically manages asset lifecycle tiers and applies automatic media caching and format optimization policies.
Deletion	Soft-deleted records and obsolete temporary session records are permanently purged from Supabase database tables using parameterized SQL queries with cascading referential integrity. Obsolete media files and product photos are permanently deleted via Cloudinary's Admin API (`UploadApi::destroy`).

Table 1: Data Lifecycle Matrix

The Data Lifecycle Matrix shows how data moves through the system from initial input to final deletion. Data is first collected through the frontend, stored securely in cloud-managed PostgreSQL (Supabase) and media cloud storage (Cloudinary), processed by serverless backend runtimes on Vercel, and accessed by authorized users. Security controls such as scoped API keys, HTTPS (TLS 1.3), transaction poolers, and database connection authentication protect the data at each stage. Retention policies ensure that data is not stored indefinitely, reducing storage costs and security risks.

Overall, the system demonstrates basic cloud data governance principles, including access control, lifecycle awareness, and structured data flow. However, improvements such as automated lifecycle policies, enhanced logging, and monitoring could further strengthen security and reliability.

Policy and Access Matrix

Identify IAM users and the policies and permissions set for them with justification on why those policies and permissions were granted.

IAM Entity	Policy Attached	Justification
Vercel Backend Service (Serverless App Runtime)	Scoped Supabase Connection: `postgres.pzsijwhcxtnaanlpwvaq` (Transaction Pooler, Port 6543) Scoped Cloudinary Credentials: `CLOUDINARY_URL` (Upload & Read access)	Allows the backend application running on Vercel serverless functions to securely execute transactional CRUD operations in Supabase and upload media to Cloudinary without exposing master administrator credentials to the client. Follows the Principle of Least Privilege by restricting database access to pooled transactions.
DevOps / Administrator User	`Vercel Project Admin`, `Supabase Project Owner` (Full DDL/DML), `Cloudinary Master Admin`	Required to provision and manage infrastructure components including Vercel edge deployments, encrypted environment variables, Supabase schema migrations, connection pooling, and Cloudinary media settings. Permissions are granted strictly for deployment, maintenance, and administrative configuration.
Auditor User (Optional / Evaluator)	`ReadOnlyAccess` (Vercel Viewer, Supabase SQL/Table Viewer Read-Only)	Allows academic panelists and auditors to monitor and evaluate cloud architecture, deployment history, database schemas, and media storage configurations without the ability to modify or delete live production resources. Ensures security oversight while preventing accidental changes.

Threat Model

Using the STRIDE model, identify possible vulnerabilities in your application and deployment strategy as well as mitigations to prevent exposure of data.

Threat	Vulnerability	Justification & Mitigation
Spoofing	**Low**	If authentication is not properly implemented, attackers could impersonate legitimate cashiers or administrators. However, enforced HTTPS on Vercel, bcrypt password hashing (12 rounds), session regeneration, and scoped database credentials mitigate credential spoofing at application and infrastructure levels.
Tampering	**Low**	Data in transit could be modified if HTTPS is not enforced between client and backend. Supabase database records or Cloudinary media could also be altered if credentials leak. Proper TLS 1.3 certificates on Vercel, CSRF token verification, parameterized queries, and atomic database transactions (`DB::transaction`) mitigate this risk.
Repudiation	**Low**	Without proper logging, users or attackers could deny performing actions (e.g., unauthorized stock adjustments, order voids, or file uploads). The application features an immutable `ActivityLog` audit trail in Supabase, while Vercel runtime logs and Cloudinary access logs provide non-repudiation.
Information Disclosure	**Low**	Exposed environment variables or open database ports could leak sensitive data. Storing sensitive credentials in Vercel's Encrypted Environment Secrets (`APP_KEY`, `DB_PASSWORD`, `CLOUDINARY_URL`), VPC connection pooling on Supabase, and role-based view masking significantly lower exposure.
Denial of Service (DoS)	**High**	High burst traffic or malicious request flooding could exhaust serverless function execution limits on Vercel or overwhelm the connection pool on Supabase's free tier. Vercel Edge DDoS protection, in-memory caching (`Cache::remember`), optimized grouped queries, and Supabase Transaction Pooling mitigate service disruption.
Elevation of Privilege	**Low**	If cloud service roles follow least privilege and no master API secrets are exposed to the client, privilege escalation risk is minimal. Server-side Laravel Role-Based Access Control (RBAC) middleware (`admin`, `inventory_manager`, `cashier`) strictly verifies permissions at every endpoint, preventing unauthorized vertical escalation.

Deployment Documentation

Create a guide on how to set up the Cloud Solution deployment. (It must also be in a README.md file in your Github repository)

Cloud Deployment Guide

This document describes the deployment architecture and step-by-step process for deploying a full-stack application to the cloud.

Architecture Overview

Frontend & Application: Laravel 11 / Tailwind CSS / Blade (deployed on Vercel)

Database: Managed Cloud PostgreSQL (deployed on Supabase)

Connection Pooler: Supabase Transaction Pooler (Port 6543)

File & Media Storage: Cloudinary Cloud Media Management

Serverless Runtime: Vercel PHP with temporary storage isolation

Security & Encryption: Enforced HTTPS (TLS 1.3) and Scoped Pooler Credentials

---

System Architecture

```
User
?
Vercel (Laravel Serverless Application)
?
Supabase (Cloud PostgreSQL Database)
?
Cloudinary (Media & File Storage)
```

---

1. Setting Up Database (Supabase PostgreSQL)

Step 1: Create Supabase Project

1. Go to https://supabase.com
2. Click **New Project**
3. Enter project name and secure database password
4. Select Region: Southeast Asia (Singapore)

Step 2: Obtain Connection Pooler String

In Supabase dashboard, click **Connect** → Select **Transaction pooler** (Port 6543) for serverless connection stability.

Step 3: Run Migrations and Seeders

```
php artisan migrate:fresh --force
php artisan db:seed --force
```

---

2. Configuring Cloudinary for Media Storage

Step 1: Obtain Cloudinary API Credentials

1. Go to https://cloudinary.com
2. From Dashboard, copy your API Environment variable (`CLOUDINARY_URL`)

Step 2: Install Package & Configure Filesystem

```
composer require cloudinary-labs/cloudinary-laravel
```

Set `FILESYSTEM_DISK=cloudinary` and register disk in `config/filesystems.php`.

---

3. Deploying Application to Vercel

Step 1: Push Project to GitHub

```
git add .
git commit -m "Production release"
git push origin main
```

Step 2: Import Project in Vercel

1. Go to https://vercel.com
2. Click **Add New Project**
3. Import your GitHub repository
4. Configure:
   Framework Preset: Other
   Root Directory: ./
   Output Directory: public